

บทที่ 4

การตรวจสอบความถูกต้องของข้อมูล โดยการคำนวณแบบ MD5

4.1 เนื้อหาโดยสังเขป

ในส่วนนี้จะเป็นการอธิบายอัลกอริทึมของ MD5 เมสเซจไดเจสซึ่งเป็นเอาอินพุตที่มีความยาวที่กำหนดเอง และมาสร้างเอาต์พุตที่มีขนาด 128 บิต จาก “fingerprint” หรือ “message digest” เป็นการคาดเดาเพื่อที่จะคำนวณที่จะสร้างข้อความ 2 ข้อความ ที่มีความสำคัญของข้อความที่คล้ายกัน หรือการสร้างข้อความที่มีจุดมุ่งหมายที่กำหนดให้เป็นไปไม่ได้ MD5จะถูกนำไปใช้ในการตรวจสอบลายมือชื่อดิจิตอล(Digital Signature) เมื่อเพิ่มข้อมูลมีขนาดใหญ่จะต้องมีการบีบอัดข้อมูลแบบปลอดภัยก่อนที่จะมีการเข้ารหัสด้วยคีย์หลัก(คีย์ที่เป็นความลับ) ภายใต้คีย์สาธารณะของระบบเข้ารหัส(Cryptosystem) อย่างเช่น อาร์เอสเอ(RSA)

MD5 ได้ออกแบบให้ใช้งานด้วยความเร็วบนเครื่อง 32 บิต แต่ถึงอย่างไรอัลกอริทึมของ MD5 ก็ไม่ต้องการตารางที่ใช้แทนจำนวนมาก(อัลกอริทึมสามารถที่จะมีโค้ดได้อย่างเร็วและกะทัดรัด)

อัลกอริทึมของMD5 เป็นส่วนที่เพิ่มเติมมาจากอัลกอริทึมของMD4 เมสเซจไดเจส ซึ่งการทำงานของ MD5 จะช้ากว่าการทำงานของ MD4 แต่MD5 จะออกแบบให้ครอบคลุมกว่า MD4 ซึ่งMD5 ถูกสร้างขึ้นเพราะว่ามีผู้สงสัยว่า MD4 จะใช้งานได้เร็วแต่มีจุดจำกัดโดยมีผู้คนมาวิจารณ์ว่ามีข้อบกพร่องเมื่อมีการใช้งาน และMD4 ถูกออกแบบให้ทำงานได้เร็วในการทำงานปกติซึ่งเสี่ยงต่อการที่จะถูกโจมตี ซึ่งMD5 ได้ออกแบบให้ทำงานได้ช้ากว่าเล็กน้อยแต่มีความปลอดภัยเป็นพิเศษ โดยเป็นการรวมเอาข้อวิจารณ์ของ MD4 มาทำการแก้ไขแล้วมาสร้างMD5 ขึ้นซึ่งMD5 จะสามารถทำงานได้บนโดเมนมาตรฐานซึ่งตัวระบุมาตรฐานของ MD5 คือ

MD5 OBJECT IDENTIFIER ::=

iso(1) member-body(2) US(840)rsadsi(113549)digestAlgorithm(2)5

4.2 คำศัพท์และความหมายของสิ่งที่กล่าวในบทนี้

ในบทนี้ เวิร์ด(Word) มีขนาด 32 บิต และ ไบต์(Byte) จะมีขนาด 16 บิต ลำดับของบิต(Bit) จะมีลักษณะแบบธรรมชาติเหมือนกับลำดับของไบต์ เมื่อการติดต่อของแต่ละกลุ่มคือ 8 บิต จะหมายถึงไบต์ด้วย high – order

ให้ x_i คือ x ลบด้วย i ถ้าส่วนที่ขีดเส้นใต้เป็นสมการเราจะตัดลัดสนใจล้อมรอบ เช่น $x_{[i+1]}$ ในทางเดียวกันเราจะใช้ $\wedge$ สำหรับการเขียนไว้ข้างบน(เอ็กซ์โพเนนเชียล) ดังนั้น x^i นั้นหมายถึง x ยก

กำลังด้วย i , สัญลักษณ์ “+” แทนการบวกเพิ่มของเวิร์ด(เช่น การเพิ่มมอดุโล -2^{32}) , $X \lll s$ การเกิดค่า 32 บิตโดยการเลื่อนบิตของ X เป็นวงกลม(การหมุน) ไปทางซ้ายจำนวน s ตำแหน่ง, $\text{not}(X)$ เป็นการกลับบิตเป็นคอมพลิเมนต์(Complement) ของ X และให้ $X \vee Y$ เป็นการออร์ (OR)ของบิตของ X กับ Y , $X \text{ xor } Y$ เป็นการเอ็กซ์คลูซีฟ-ออร์(xor) ของ X กับ Y และ XY เป็นการแอนด์(AND) บิตของ X และ Y

4.3 MD5 อัลกอริทึม (MD5 Algorithm)

เราจะเริ่มโดยการมีข้อความ b เป็นอินพุต(Input) เราต้องการหาเมสเสจดีเจส(Message Digest) ข้อความ b จะไม่เป็นจำนวนเต็มที่เป็นลบ b จะต้องเป็น 0 ไม่ต้องการผลคูณด้วย 8 และไม่ต้องการเกณฑ์ขนาดใหญ่เราลองจินตนาการบิตของข้อความที่เขียนลงจะเป็นดังนี้

$$m_0 m_1 \dots m_{[b-1]}$$

ปฏิบัติตาม 5 ขั้นตอนดังต่อไปนี้ในการคำนวณเมสเสจดีเจสของข้อความ

ขั้นตอนที่ 1 การเพิ่มบิตแพดดิ้ง (Append Padding Bits)

ข้อความเป็นแพด(ส่วนขยาย) ซึ่งมีความยาว(เป็นบิต)ที่มอดุโล 512 แล้วเท่ากับ 448 หากไม่ถึงหรือเกินให้เพิ่มเติมแพดดิ้งเข้าไป ดังนั้นข้อความที่จะขยายจะต้องมีอย่างน้อย64บิตของผลคูณของ 512 จะทำการแพดดิ้งเสมอ ถ้าความยาวของข้อความจริงหารด้วย 512 เท่ากับ 448 การทำแพดดิ้งจะทำแบบนี้ เป็นบิต 1 เดียว เป็นตัวเพิ่มเติมของข้อความและบิตเป็น 0 เป็นการเพิ่มเติมข้อความในบิตแพดดิ้งให้หารด้วย 512 มีค่าเท่ากับ 448 ดังนั้นบิตที่ต้องขยายน้อยสุด 1 บิตและขยายมากที่สุด 512 บิต

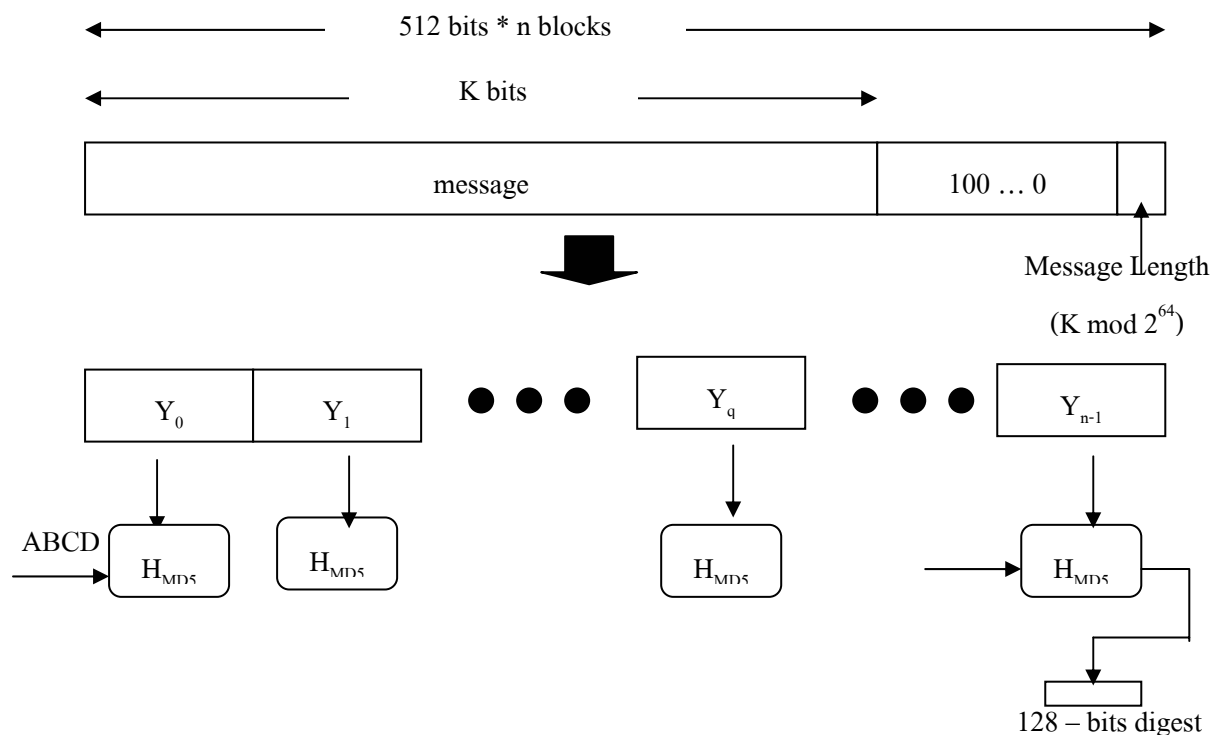
ขั้นตอนที่ 2 เพิ่มเติมความยาว (Append Length)

64 บิตของ b (ความยาวของข้อความก่อนการเพิ่มบิตแพดดิ้ง) เป็นการขยายขั้นตอนก่อนหน้านี้ ดังนั้น b ต้องไม่น่ามีค่ามากกว่า 2^{64} เมื่อมีแค่ 64 ตำแหน่งที่ใช้(จะมีการขยาย 32 บิตเวิร์ด และขยายบิตเวิร์ดค่าที่ตรงกับค่าก่อนหน้านี้) ที่จุดนี้เป็นผลของข้อความ(ด้วยการรวมบิตแพดกับ b) ความยาวที่แน่นอนจะเป็นของผลคูณ 16(32 บิต)เวิร์ด โดย $M[0 \dots N-1]$ แทนผลของข้อความ เมื่อ N เป็นผลคูณของ 16

ขั้นตอนที่ 3 การกำหนดค่าเริ่มต้นเมสเสจดีเจสบัฟเฟอร์ (MD Buffer)

บัฟเฟอร์จะมี 4 ส่วน(A, B, C, D) ที่ใช้ในการคำนวณเมสเสจดีเจสแต่ละตัวของ A, B, C, D เป็นรีจิสเตอร์ 32 บิต โดยรีจิสเตอร์เหล่านี้จะมีค่าเริ่มต้นจะมีค่าเริ่มต้นในฐานสิบหก

เวิร์ด A	: 01	23	45	67
เวิร์ด B	: 89	ab	cd	ef
เวิร์ด C	: ef	dc	ba	98
เวิร์ด D	: 76	54	32	10



รูปที่ 4 – 1 แสดงการคำนวณแบบ MD5

ขั้นตอนที่ 4 กระทำข้อความใน 16 เวิร์ดบล็อก (Process Message in 16 – Word Block)

เริ่มด้วยการกำหนด 4 ฟังก์ชันช่วยโดยเอา 32 บิตเวิร์ด 3 ตัว และมาสร้างเอาท์พุต 32 บิตเวิร์ด 1 ตัว

$$F(X, Y, Z) = XY \vee \text{not}(X)Z$$

$$G(X, Y, Z) = XZ \vee Y\text{not}(Z)$$

$$H(X, Y, Z) = X \text{ xor } Y \text{ xor } Z$$

$$I(X, Y, Z) = Y \text{ xor } (X \vee \text{not}(Z))$$

ในแต่ละตำแหน่งบิตที่กระทำ F มีเงื่อนไขดังนี้ : ถ้า X เป็นจริง จะได้ Y ถ้าเป็นเท็จจะได้ Z ฟังก์ชัน F จะกำหนดการใช้ + การแทนที่ของ $\vee$ (or) เพราะ XY และ $\text{not}(X)Z$ จะไม่เป็น 1 ในตำแหน่งบิตเดียวกัน ที่สนใจก็คือ ถ้าบิตของ X, Y, และ Z เป็นอิสระต่อกันและตรงไปตรงมา ดังนั้นแต่ละบิตของฟังก์ชัน $F(X, Y, Z)$ จะเป็นอิสระต่อกัน

ฟังก์ชัน G, H, และ I จะมีลักษณะการทำงานคล้ายกับฟังก์ชัน F ซึ่งการทำงานจะทำแบบบิตไวด์ พาราเลล (Bitwise parallel) ที่จะสร้างเอาท์พุตจากบิตของ X, Y, และ Z แบบที่ว่าถ้าบิตที่ตรงกันของ X, Y, และ Z จะเป็นอิสระต่อกัน แล้วแต่ละบิตของฟังก์ชัน $G(X, Y, Z)$, $H(X, Y, Z)$ และ $I(X, Y, Z)$ จะเป็นอิสระต่อกัน เมื่อฟังก์ชัน H เป็นบิตไวด์ เอ็กซ์ออ์ (xor) หรือ พาริตี (parity) ฟังก์ชันของอินพุต ในขั้นนี้ จะใช้ตารางขนาด 64 เอลเมนต์ (element) $T[1 \dots 64]$ สร้างฟังก์ชัน เมื่อ $T[i]$ โดย I เป็นเอลเมนต์ของ

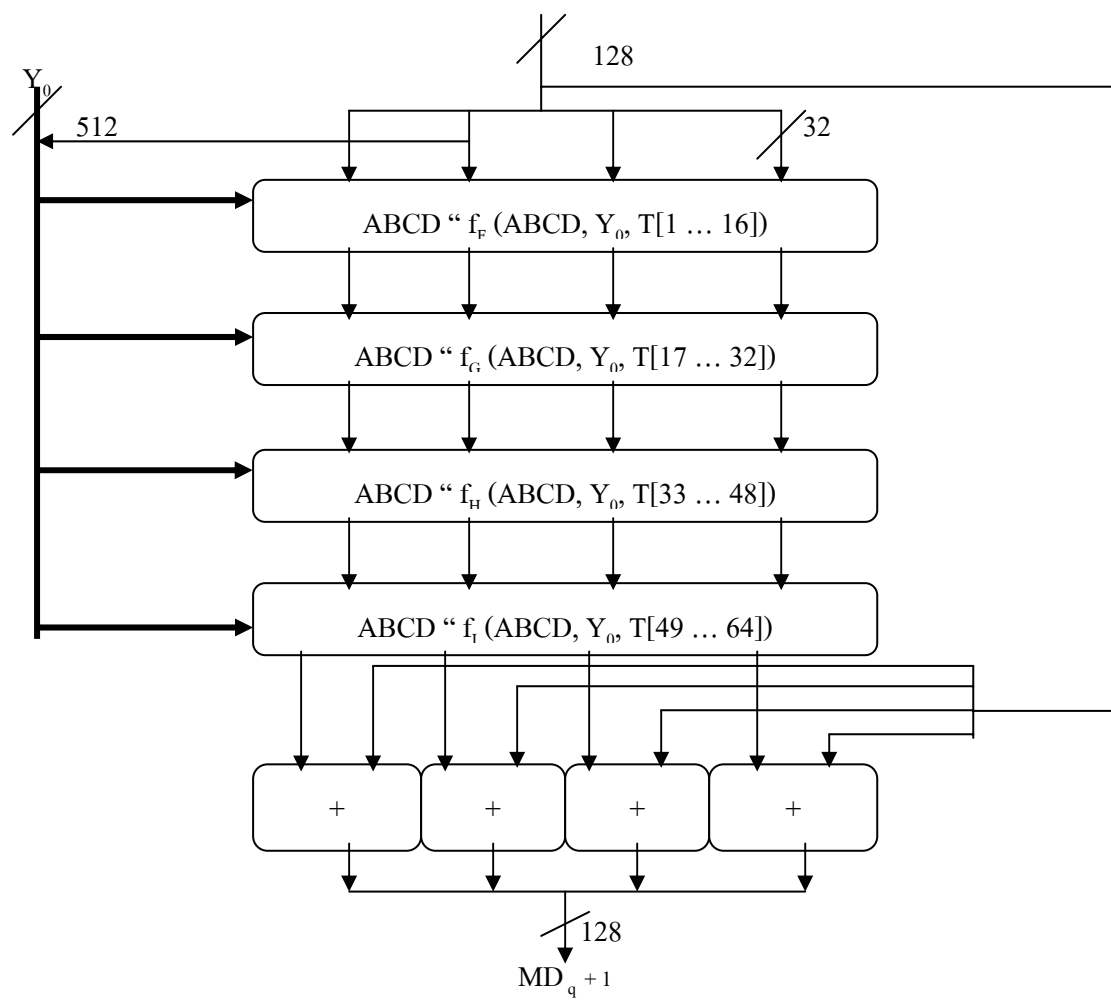
ตารางซึ่งเท่ากับจำนวนเต็มส่วนของ 4294967296 ครั้ง $\text{abs}(\sin(i))$ เมื่อ i มีค่าเป็นในเรเดียน ของเอเลเมนต์
ของตารางจะกล่าวในส่วนต่อไป ซึ่งการทำงานจะทำตามนี้

```
/*การกระทำแต่ละ 16 เวิร์ดบล็อก */
For I = 0 to N/16-1 do
/* สำเนาจากบล็อก I ไปยังบล็อก X */
For j = 0 to 15 do
Set X[j] to M[I*16+j]
End /*of loop on j*/
/* เก็บค่า A เท่ากับ AA, Bเท่ากับ BB, C เท่ากับ CC, และ D เท่ากับ DD*/
AA = A
BB = B
CC = C
DD = D
/* รอบที่ 1 */
/* ให้ [abcd k s I] ใช้แทนโอเปอเรชั่น  $a = b + ((a + F(b, c, d) + X[k] + t[I]) \lll s)$  */
/* ทำตาม 16 โอเปอเรชั่น */
[ABCD 0 7 1] [DABC 1 12 2] [CDAB 2 17 3] [BCDA 3 22 4]
[ABCD 4 7 5] [DABC 5 12 6] [CDAB 6 17 7] [BCDA 7 22 8]
[ABCD 8 7 9] [DABC 9 12 10] [CDAB 10 17 11] [BCDA 11 22 12]
[ABCD 12 7 13] [DABC 13 12 14] [CDAB 14 17 15] [BCDA 15 22 16]
/*รอบที่ 2. */
/* ให้ [abcd k s i] ใช้แทนโอเปอเรชั่น  $a = b + ((a + G(b,c,d) + X[k] + T[i]) \lll s).$  */
/* ทำตาม 16 โอเปอเรชั่น */
[ABCD 1 5 17] [DABC 6 9 18] [CDAB 11 14 19] [BCDA 0 20 20]
[ABCD 5 5 21] [DABC 10 9 22] [CDAB 15 14 23] [BCDA 4 20 24]
[ABCD 9 5 25] [DABC 14 9 26] [CDAB 3 14 27] [BCDA 8 20 28]
[ABCD 13 5 29] [DABC 2 9 30] [CDAB 7 14 31] [BCDA 12 20 32]
/* รอบที่ 3. */
/* ให้ [abcd k s t] ใช้แทนโอเปอเรชั่น  $a = b + ((a + H(b,c,d) + X[k] + T[i]) \lll s).$  */
/* ทำตาม 16 โอเปอเรชั่น */
[ABCD 5 4 33] [DABC 8 11 34] [CDAB 11 16 35] [BCDA 14 23 36]
[ABCD 1 4 37] [DABC 4 11 38] [CDAB 7 16 39] [BCDA 10 23 40]
```

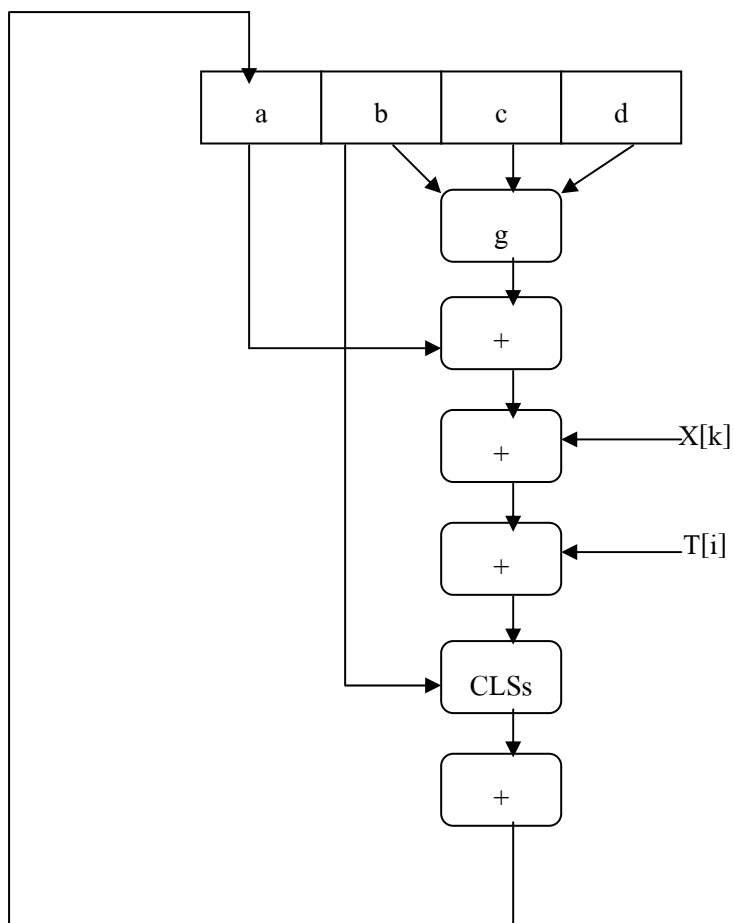
```

[ABCD 13 4 41] [DABC 0 11 42] [CDAB 3 16 43] [BCDA 6 23 44]
[ABCD 9 4 45] [DABC 12 11 46] [CDAB 15 16 47] [BCDA 2 23 48]
/* รอบที่ 4. */
/* ให้ [abcd k s t] ใช้แทนโอเปอเรชั่น  $a = b + ((a + I(b,c,d) + X[k] + T[i]) \lll s)$ . */
/* ทำตาม 16 โอเปอเรชั่น*/
[ABCD 0 6 49] [DABC 7 10 50] [CDAB 14 15 51] [BCDA 5 21 52]
[ABCD 12 6 53] [DABC 3 10 54] [CDAB 10 15 55] [BCDA 1 21 56]
[ABCD 8 6 57] [DABC 15 10 58] [CDAB 6 15 59] [BCDA 13 21 60]
[ABCD 4 6 61] [DABC 11 10 62] [CDAB 2 15 63] [BCDA 9 21 64]
/* เมื่อการเพิ่มตามนี้ (เป็นการเพิ่มแต่ละตัวของรีจิสเตอร์ทั้ง 4 ตัว โดยค่าที่ก่อนที่บล็อกรจะทำการ
เริ่มทำงาน)*/
A = A + AA
B = B + BB
C = C + CC
D = D + DD
end /* รอบของ i */

```



รูปที่ 4 – 2 กระบวนการทำในหนึ่งบล็อกของ MD5



รูปที่ 4 – 3 แสดงการทำหนึ่งโอเพอร์เรเตอร์ของ MD5 [abcd k s i]

โดยที่

$$a \leftarrow b + \text{CLSs}(a + g(b, c, d) + X[k] + T[i]) \text{ และ}$$

Round	Primitive function g	G(b, c, d)
F _F	F(b, c, d)	(b . c) v (not(b) . d)
F _G	G(b, c, d)	(b . d) v (c . not(d))
F _H	H(b, c, d)	$b \oplus c \oplus d$
F _I	I(b, c, d)	$c \oplus (b . \text{not}(d))$

ตารางที่ 4 – 1 เงื่อนไขของฟังก์ชัน

ขั้นตอนที่ 5 เอาท์พุต(output)

เมสเสจสคิเจส จะมีเอาท์พุตเป็น A, B, C, D โดยเริ่มที่ไบนารีต่ำของ A และจบด้วยไบนารีสูงของ D

4.4 ข้อแตกต่างระหว่าง MD4 กับ MD5

ข้อแตกต่างระหว่าง MD4 และ MD5 มีดังนี้

1. มีแค่ 4 รอบ สำหรับการเพิ่ม
2. ในแต่ละขั้นตอนปัจจุบันจะมีการเพิ่มด้วยค่าคงที่เฉพาะ
3. ฟังก์ชัน g ในรอบที่ 2 จะเปลี่ยนจาก $(XY \vee XZ \vee YZ)$ เป็น $(XZ \vee Y \text{ not}(Z))$ และทำเป็นสัดส่วนน้อยที่สุดของ g
4. แต่ละขั้นตอนปัจจุบันจะเพิ่มด้วยค่าจากผลของขั้นตอนที่แล้ว เป็นการนำเสนอที่รวดเร็ว เรียกว่า “avalanche effect”
5. แบบที่อินพุตเวิร์ดที่เข้ามาในรอบที่ 2 รอบที่ 3 จะเปลี่ยน ทำเป็นแพทเทิร์นที่น้อยเหมือนตัวอื่น
6. การเลื่อนรวมในแต่ละรอบจะมีประมาณน้อยสุดและเร็วกว่า “avalanche effect” การเลื่อนในรอบที่แตกต่างจะไม่ซ้ำ

4.5 สรุป

แมสเสจสคิเจสลำดับที่อัลกอริทึมเป็นเครื่องมืออย่างง่าย และเป็นการกำหนดฟังก์ชันปรีนเปื่องดัน หรือแมสเสจสคิเจสของข้อความที่มีความยาวไม่แน่นอน ซึ่งจะทำให้การเดาค่าที่ค่าที่แตกต่างที่เกิดขึ้นของสองข้อความมีลักษณะที่เหมือนกันของแมสเสจสคิเจสอยู่บนแบบแผนของ 2^{64} โอเปอร์เรเตอร์ และความแตกต่างที่เกิดขึ้นของข้อความที่แมสเสจสคิเจสบนแบบแผนของ 2^{128} โอเปอร์เรเตอร์ MD5 ยังระมัดระวังและวิเคราะห์ถึงจุดอ่อน อย่างไรก็ตาม อัลกอริทึมใหม่ที่เกี่ยวข้องและอัลกอริทึมใหม่ที่มีความเกี่ยวข้องกับตัวใหม่และการวิเคราะห์ความปลอดภัยเป็นการนำเสนอเฉพาะคอร์สเท่ากับเป็นข้อเสนอต่อไปตามลำดับ